

Scope and related work

Positioning

Machine learning for big code surveys cover naturalness, representation learning, and task taxonomy [allamanis2018survey]. COGANT contributes **infrastructure**: a documented IR, pipeline, and export contract rather than a new benchmark or model architecture.

Graph neural networks unify message-passing frameworks for relational data [wu2020comprehensive; scarselli2009graph]. Although COGANT's primary output is the Active Inference Institute's Generalized Notation Notation, its program graph can be materialised as optional tensor views compatible with these frameworks (PyG, DGL) so that graph-neural-network model papers can cite a specific tensor layout.

Code property graphs and related security-oriented graph constructions show how to merge syntactic and semantic edges for analysis [yamaguchi2014modeling]. COGANT's program graph is cognate but emphasizes **ML export**, confidence scoring, and multi-stage IR refinement.

Related tool categories

- ▶ **Compiler and LLVM IRs** — richer semantics, heavier toolchain; COGANT favors lightweight extraction for dataset building.
- ▶ **SCA and linters** — rule enforcement focus; COGANT focuses on graph generation for downstream learning.
- ▶ **Neural code models** (code2vec, CodeBERT, etc.) — often consume tokens or AST paths; COGANT supplies **explicit program graphs** that graph-neural-network-centric research can ingest directly.

Program analysis for ML

Several systems address the intersection of program analysis and machine learning that COGANT operates in:

code2seq and code2vec [alon2019code2vec] represent programs as sets of AST paths between terminal nodes. These path-based representations are effective for method naming and code summarization but discard the full graph topology that graph-neural-network-based models exploit. COGANT preserves the complete program graph — including control-flow and data-flow edges — and can export it in tensor form, enabling downstream architectures that reason over richer relational structure.

Gated Graph Neural Networks (GGNN) [li2016gated] introduced gated recurrent propagation for graph-structured program representations, demonstrating strong results on variable misuse detection and other code tasks. COGANT's export contract (PyG Data objects with typed `edge_index` and `edge_attr`) is directly compatible with GGNN-style models; the kind and role

Active inference and program behavior

The state-space IR in COGANT's pipeline (states, actions, transitions, observations) shares structural parallels with **active inference** formulations [friston2010free], where an agent maintains beliefs about hidden states and selects actions to minimize prediction error. In the program analysis context, the “agent” is the analysis pipeline itself: it observes code artifacts, maintains beliefs about program behavior (the state-space model), and refines those beliefs as new evidence (dynamic traces, coverage data) arrives.

This connection is analogical: the `ConfidenceModel` in `../cogant/py/cogant/translate/confidence.py` aggregates evidence and penalties in a way that suggests belief revision, but it is not a Bayesian posterior. Future work could formalize a tighter link by casting rule application as variational inference, where a fixpoint would represent an approximate posterior over program semantics.

Boundaries

COGANT does not subsume formal verification, interactive theorem proving, or full interprocedural pointer analysis unless implemented as explicit future stages. The SPEC marks Rust acceleration and additional parsers as staged; the manuscript should be read together with that table for up-to-date scope.

Forward compatibility

Promoting COGANT into ../../../../projects/ integrates manuscript PDF rendering with the template's validation gates. Cross-references in this folder use paths **relative to these Markdown files** (for example ../../cogant/docs/) so links stay stable when the tree moves.